FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

FH Aachen

Bachelor's Thesis

in partial fulfillment of the requirements for the degree of
B. Sc. in Computer Science

AI Agents in Software Development: Best Practices for Structured Human-Agent Collaboration

Fakih Lanjri Houdaifa

Matriculation Number 3574047 Aachen,

First Examiner:

Second Examiner:

Declaration

I hereby declare that I have written this thesis independently and have not used any sources other than those indicated in the bibliography. Passages that have been taken verbatim or in substance from published or as yet unpublished sources are identified as such. The drawings or illustrations in this thesis were created by myself or are provided with an appropriate reference to their source. This thesis has not been submitted in the same or similar form to any other examination authority.

Aachen,

Contents

1 Introduction

1.1 Background

Large Language Models have moved a long way from being simple autocomplete engines for code. Today, tools built on top of them act as genuine coding agents: they can read a set of requirements, plan an implementation, write entire features, generate their own tests, and reason, at least to some degree, about the architecture of the software they are building. Modern coding agents can be integrated directly into conventional development environments, allowing developers to hand off substantial parts of the implementation work to an AI agent while still reviewing and supervising what it produces.

More recently, agent-first development environments have emerged that extend this concept further. Rather than relying on a single agent working through a linear sequence of tasks, these environments can support the orchestration of several specialized agents, each responsible for different parts of the development workflow, from planning and architectural design to implementation and verification.

This shift changes the nature of software development in a subtle but important way. In traditional development, the person writing the code is also the one reasoning about it. When an AI agent takes over the implementation, a layer is inserted between the idea and the code: the agent interprets instructions, makes its own architectural choices, and produces artifacts that a human then has to review, correct, or approve. How good the resulting software turns out to be depends not only on how capable the underlying model is, but just as much on how the collaboration between human and agent is set up in the first place — how precise the prompts are, whether any architectural boundaries are defined at all, and what kind of review process, if any, the agent's work has to pass.

1.2 Motivation

This thesis grew out of a practical project in which software was built from the ground up using AI coding agents, across two different development environments that each embody a different philosophy of working with an agent.

The first part of the project was carried out with Claude Code inside Visual Studio Code. Development here followed a deliberately loose approach: the agent was asked to implement features from scratch with very little structural guidance, beyond a general description of the software idea, the functional and non-functional requirements, and the core modules of the web application. The goal was to observe how an AI agent behaves when given substantial implementation freedom within a single-agent, conversational workflow — and where exactly that freedom starts to cause problems, whether in the form of inconsistency, architectural drift, or small but consequential implementation mistakes.

What came out of that first part shaped the second one directly. For the second part, development moved to Google Antigravity, an agent-first development environment organized around orchestrating multiple agents rather than interacting with a single one. Instead of assuming that better software simply requires a more capable model, this second phase focused on something different: the architectural rules, review steps, and development discipline imposed on the agents themselves. A more structured methodology was introduced for this purpose, built around an AI Development Charter, a three-agent setup (an Architect, an Implementer, and a Post-Implementation Architecture Review Agent), mandatory design reviews before any code was written, and a formal testing strategy — all of it designed to make deliberate use of the environment's support for parallel, role-specialized agents working together, rather than one agent working through a single, linear thread of tasks.

The motivation behind this thesis is therefore twofold. First, it aims to document and compare, in concrete terms, what changes when moving from an unstructured, single-agent approach to a structured, multi-agent approach in AI-assisted software development. Second, it aims to distill from that comparison a set of best practices for human-agent collaboration that hold up beyond the specific tools used here, and that can inform how developers approach this kind of work more generally.

1.3 Problem Statement

AI coding agents are being increasingly adopted in both industry and academic settings. However, systematic understanding of how the structure of human-agent collaboration — rather than solely the capability of the underlying model — influences the quality, architectural soundness, and reliability of resulting software remains limited. Existing research on AI-assisted software development has often focused on model capabilities, benchmarks, or individual code-generation tasks, while broader aspects of the development process —

including requirements interpretation, architectural planning, iterative implementation, testing, review, and documentation maintenance — require further investigation.

Several specific challenges motivate the work presented in this thesis:

1. **Limited structural comparison.** Existing work provides limited empirical comparison between unstructured, prompt-driven interaction with AI coding agents and more structured, architecture-first development approaches, particularly when both approaches are examined within the same software project.

2. **Unclear impact of architectural constraints.** It remains unclear to what extent architectural rules, mandatory design reviews, and the separation of responsibilities across specialized agents can improve the quality and consistency of AI-generated software compared with a single agent operating with fewer structural constraints.

3. **Gaps in testing and validation.** AI coding agents can generate tests that execute successfully without necessarily providing sufficient validation of the intended behavior. This raises the question of how testing and verification should be structured when substantial parts of both implementation and test generation are delegated to AI agents.

4. **Documentation drift.** As AI agents modify software iteratively, maintaining consistency between the implementation and related specifications, such as OpenAPI documents, becomes an important maintainability challenge.

5. **Limited consolidated guidance.** Despite the growing adoption of AI coding agents, empirically grounded guidance on how developers should structure human-agent collaboration across prompting, architecture, review, testing, and documentation remains fragmented.

1.4 Objectives of the Thesis

The aim of this thesis is to systematically document, analyze, and compare two contrasting approaches to AI-agent-assisted software development — one unstructured and one structured — based on a practical project, and to derive from this comparison concrete, actionable best practices for human-agent collaboration in software engineering.

More specifically, this thesis sets out to:

1. **Document the unstructured approach (Part 1).** Describe how the software was developed using a single-agent workflow with minimal architectural guidance, and analyze the resulting architectural limitations, the influence of prompt precision, and the extent to which human supervision was required.

2. **Document the structured approach (Part 2).** Describe the design and implementation of a structured, multi-agent development methodology, including the AI Development Charter, parallel role-specialized agents, mandatory design reviews, the three-tier testing strategy, and the automation of OpenAPI specifications to prevent documentation drift.

3. **Evaluate each approach.** Define evaluation criteria and assess each development approach with respect to code quality, test validity, architectural consistency, documentation consistency, and development efficiency.

4. **Compare the two approaches.** Conduct a direct cross-approach comparison to determine how the structured methodology affected the limitations observed during the unstructured development phase, including differences in feature implementation, code and test quality, architectural consistency, and documentation reliability.

5. **Derive best practices.** Bring the findings together into a set of best practices for human-agent collaboration, covering prompt precision, the role of human oversight, architectural guardrails, recommendations for multi-agent workflows, and strategies for preventing documentation drift.

Taken together, these objectives position the thesis as both an empirical case study of AI-agent-assisted software development and a practically oriented source of recommendations that developers and organizations can draw on when structuring their own collaboration with AI coding agents.

2 Foundations

2.1 Large Language Models in Software Engineering

Large Language Models (LLMs) are, at their core, autoregressive sequence generators: given a sequence of tokens as input, the model predicts a probability distribution over the next token and generates output by repeatedly sampling from this distribution. Nearly all current LLMs are built on the Transformer architecture, which underlies their ability to generate long, coherent sequences of text — including code.

When applied to source code, this generative mechanism is unchanged in principle: code is treated as a token sequence like any other. Just as a language model predicts the next word of a sentence from the words that came before it, a code-generating model predicts the next token of a program from the tokens that came before it — where a token may be a keyword, a variable name, a punctuation mark, or a fragment of one. Given the beginning of a function signature such as `def calculate_total(`, the model does not "know" what a function is in any formal sense; it predicts, token by token, that a parameter name is likely to follow, then a closing parenthesis, then a colon, then an indented block — because sequences with this shape occurred overwhelmingly often in the code it was trained on. The model's fluency with syntax, naming conventions, and common patterns is therefore a learned statistical regularity, not an explicit understanding of a programming language's grammar or semantics.

The practical capability of LLMs applied to code has developed considerably since general-purpose language models were first adapted to this domain. Early applications were largely limited to local code completion: a model would suggest the rest of a line, or perhaps the body of a short function, based only on the few lines immediately surrounding the cursor — closer to a smarter version of an IDE's autocomplete than to an independent programmer. A notable step beyond this was Codex, a GPT-based model that started as a general-purpose language model and was then further trained — fine-tuned — specifically on publicly available code from GitHub, which made it capable of writing Python code from natural-language docstrings alone. This meant the model could take nothing but a function signature and a description of the intended behavior, such as

```python
def is_palindrome(s: str) -> bool:
    """Check if a string is a palindrome, ignoring
    case and non-alphanumeric characters."""
```

and generate a complete, working implementation beneath it — predicting, token by token, what code plausibly follows such a docstring, based on patterns learned from the enormous number of similar functions it had seen during training. On the HumanEval benchmark introduced alongside it, Codex solved 28.8% of hand-written programming problems from their descriptions alone when generating a single attempt. When the model was instead allowed to generate up to 100 candidate solutions per problem and only one needed to pass, its success rate rose to 70.2% — an early illustration that a single generated output is often unreliable on its own. This shift — from finishing a line to generating a working implementation from a description of intended behavior — is what makes it possible, in principle, to prompt a model with a feature description and receive an entire piece of software back, rather than a suggestion for what to type next. It is precisely this capability that current coding agents build upon.

These structural properties belong to the model itself, considered independently of any system built around it. A "call" to an LLM — a single request that sends in some text and receives generated text back — has no access to a filesystem, cannot execute code, and does not persist anything once it finishes responding, unless additional tooling is deliberately built around it to provide these capabilities. The limitations described below apply to the model in this bare form:

- **Fixed context window.** A model can only condition its output on a bounded amount of text supplied in a single call — its context window. Imagine asking a developer to fix a bug in one file of a large application, but showing them only that one file and none of the others it depends on: they might produce a change that looks correct in isolation while silently breaking an assumption made somewhere else in the codebase, simply because they never saw it. An LLM faces the same problem whenever a project's relevant files, conventions, or history exceed what fits in a single call. This limits the context available to the model when making decisions, can prevent it from reasoning across an entire unit of code at once, and means the model simply cannot process code that does not fit within the window unless the relevant information is explicitly and selectively supplied as input. This limitation is well documented specifically for coding LLMs: the difficulty of maintaining context over long sequences of code is considered a significant technical limitation of LLMs in code generation, since the functionality and correctness of code often depends

on information that lies outside of what fits in a single window. A single LLM call therefore cannot be expected to hold an entire codebase, its full change history, or an extensive set of project-specific conventions in mind at once.

- **No persistent memory across calls.** Each generation is, by default, stateless: the model does not retain anything from a previous call unless that information is explicitly re-supplied as part of the current prompt. Concretely, if a developer asks a model to implement a login feature, and in a later, separate call asks it to write tests for "the login feature I just described," the model has no memory of the earlier conversation to draw on — from its perspective, "the feature I just described" refers to nothing at all, unless the original description is pasted back in. Any continuity across a multi-step development task must therefore be actively maintained by whatever system is orchestrating the calls, not by the model itself.

- **No inherent mechanism for verification.** An LLM generates output that is statistically plausible given its training and the supplied context, but it has no built-in means of executing that output, checking it against a specification, or confirming that it behaves as intended. This is a specific instance of a broader, well-studied phenomenon in LLM research known as hallucination — the generation of content that is fluent and syntactically well-formed but factually inaccurate or unsupported by the available evidence. Applied to code, this means a model can generate a function that reads as entirely plausible — correct indentation, sensible naming, no syntax errors — while still returning the wrong result for certain inputs, calling a method that doesn't exist, or silently failing to handle an edge case the requirement demanded. The model has no way of catching this itself, since doing so would require actually running the code and comparing its behavior against the specification — a step the model itself cannot perform. The same applies to generated tests: a test that is syntactically valid and executes without error is not automatically a test that meaningfully validates the intended behavior.

Taken together, these properties mark the boundary of what a bare LLM call can be expected to do: it provides a generative and reasoning capability over token sequences, including code, but has no native ability to interact with a filesystem, execute code, retain state across multiple turns, or verify the correctness of its own output. Any system that requires these capabilities — as agentic coding tools do — must supply them externally, layered on top of the model rather than as an inherent feature of it. This distinction motivates the notion of a coding agent, introduced in the following section.

2.2 AI Coding Agents and Agentic Workflows

A bare LLM call, as discussed above, cannot access a filesystem, execute code, retain memory across calls, or verify its own output. A coding agent is what results when a system is deliberately built around an LLM to supply exactly these missing capabilities. The LLM still provides the underlying reasoning and text-generation capability, but it is now embedded in a loop that lets it observe its environment, take concrete actions in it, and adjust its next step based on what happens — rather than producing a single block of text and stopping. A common way of describing this is in terms of three capabilities that, together, enable autonomous behavior: planning, memory, and tool use.

From single generation to an iterative loop. A widely used way of structuring this loop, introduced under the name ReAct, has the model cycle repeatedly through three types of output instead of producing a single final answer: a reasoning step describing its understanding of the problem and its plan, an action it takes to gather information or affect its environment, and an observation of the result of that action, which then feeds into the next reasoning step. Instead of a plain LLM call, which receives an input and produces one finished answer, an agent structured this way produces a reasoning step, then an action, then reads back an observation of what that action actually returned — and repeats this cycle, each round informed by the last, until it judges the task complete.

Applied to a coding task, this loop might look concretely as follows. Given the instruction **"add input validation to the registration form,"** an agent does not simply generate a finished file in one step:

1. Reasoning: the agent reasons that it first needs to see the existing form implementation before it can change it.
2. Action: it takes the action of reading that file.
3. Observation: it observes the file's actual content — the current fields, structure, and existing logic.
4. Reasoning: based on that observation, it reasons about where validation logic should be added.
5. Action: it takes the action of writing the change to the file.
6. Observation: it runs the project's test suite as a further action and observes whether the tests pass.
7. Reasoning: if a test fails, it reasons about why, and the cycle continues — reading further code, revising the change, re-running tests — until the observation confirms the task is complete.

Each individual step in this loop is still, underneath, a single LLM call — stateless, bounded by a context window, with no ability to verify anything on its own. What the agent adds is the surrounding structure that decides what to feed into the next call, based on what came back from the previous one, so that seven or more individual calls end up behaving like one continuous, self-correcting attempt at the task.

Tool use. The "actions" in this loop are only possible because the agent is given access to tools: concrete operations it can invoke, such as reading or writing a file, running a shell command, executing a test suite, or querying an external service. A representative example of teaching a model to use tools is Toolformer, a model trained to decide which external APIs to call, when to call them, what arguments to pass to them, and how to incorporate their results back into what it generates next. A coding agent applies the same underlying idea to a developer's toolbox instead of a calculator or a search engine: instead of an API for looking up a fact, the tools are things like "open this file," "apply this change," or "run this command and report what happened." This is precisely what compensates for the limitations of a bare model call — it has no way to read a file it wasn't shown or to check whether code it wrote actually runs, but an agent with file-reading and code-execution tools can do both, by making additional, purpose-specific calls to the model in between.

Memory and persistent context. An LLM, left on its own, does not remember anything between separate calls unless that information is explicitly resupplied. A coding agent addresses this not by giving the model memory it does not have, but by having the surrounding system track relevant state — which files have been read or changed, what the original task was, what has already been tried — and re-inserting the necessary parts of that state into each new call. From the developer's point of view, the agent appears to "remember" earlier parts of a multi-step task; underneath, this is the agent's surrounding system doing the remembering on the model's behalf, one call at a time.

Coding agents specifically. Because software engineering tasks are inherently complex, modular, and often require coordinating multiple steps, they were among the first domains where this kind of agent architecture was applied in practice. A coding agent built this way can be handed a task description substantially larger than a single function — implement a feature, fix a reported bug, refactor a module — and is expected to work through it the way the loop above illustrates: inspecting the relevant parts of a codebase, making a change, running the project's own tests or build tools to check the result, and revising the change if that check fails, without a human needing to manually restate context or re-supply files at every step.

Single-agent versus multi-agent workflows. So far, this section has described an agent as a single loop: one model, cycling through reasoning, action, and observation to work

through a task. Not every coding agent is structured this way. Some development environments instead coordinate several such loops at once, each running with a different role, a different set of tools, or a different part of the task — for instance, one agent responsible for planning an implementation, and a separate one responsible for carrying it out. In this arrangement, the agents do not merely take turns; each is a full reasoning-action-observation loop in its own right, and the environment is responsible for passing relevant output from one agent to another rather than leaving a single model to manage every concern at once.

3 Methodology

3.1 Structure of the Practical Project (Two Sub-Projects)

3.2 Tools and Models Used

3.3 Evaluation Criteria

4 Part 1: Unstructured Development

4.1 Initial Situation and Objectives

4.2 Prompting Without a Structural Approach

4.3 System Prompt Context

4.4 Development of the First Feature (Login/Registration)

4.5 Test Suites and Test Case Generation

4.6 Architectural Analysis of the Generated Software

4.7 Observations: Prompt Precision, Human Oversight, Architectural Limitations

4.8 Critical Problems and the Test Failure Analysis Principle”

5 Part 2: Structured Approach (AntiGravity)

5.1 Motivation for a Structured Approach

5.2 AI Development Charter and Architectural Rules

5.3 Parallel Agent Architecture (Architect / Implementer / Post-Implementation Review Agent)

5.4 Design Reviews Before Coding (implementation_plan.md)

5.5 Testing Strategy: Force Requirement Validation & Three-Tier Testing

5.6 Project Knowledge Files

5.7 OpenAPI Specification Automation and Avoiding Documentation Drift

5.8 Identification of Architectural Problems and Weaknesses

6 Comparison of Both Approaches

6.1 Comparison: Part 1 vs. Part 2

6.2 Feature Diversity

7 Evaluation

7.1 Evaluation Criteria and Methodology

7.2 Quality of Generated Code

7.3 Quality and Validity of Generated Tests

7.4 Architectural Conformance and Documentation Consistency

7.5 Efficiency and Effort (Human vs. Agent)

7.6 Summary Assessment: Part 1 vs. Part 2

8 Best Practices for Human-Agent Collaboration

8.1 Importance of Prompt Precision and Context Design

8.2 Role of Human Oversight (Human-in-the-Loop)

8.3 Architectural Guardrails for AI Agents

8.4 Recommendations for Multi-Agent Workflows

8.5 Avoiding Documentation Drift

9 Discussion

9.1 Opportunities and Limitations of AI Agents in Software Development

9.2 Implications for Practice

10 Conclusion and Outlook

Bibliography